

# COMP101

## Introduction to Programming

### 2025-26

## Lecture-19

### Function Parameterisation

#### (Arguments and Parameters )

# Passing data to functions

- Calling a function using empty brackets means the function executes only with the data that is inside it (its indented code)
- We can call a function and put data in the brackets – this data is ‘passed’ to the function and means the function can execute with different data each time it is called
- This is ‘parameterisation’ – calling a function and passing data to it

# Data passing mechanism

#count length of a word

```
def length_of_word (word):  
    length = (len(word))  
    print ("Word length = ", length)  
    return
```

```
def main():  
    word = input ("Enter a word: ") test  
    length_of_word (word)
```

#start

```
main()
```

main() calls def main():

def main(): word is a **variable** assigned string 'test'

length\_of\_word (word) calls function length\_of\_word  
It calls using using ( word ) as the '**argument**' assigned string 'test'

(word) as the **argument** is passed to

length\_of\_word (word)

where word is the '**parameter**' to the function: word has value 'test'

length\_of\_word now uses **parameter** (word) to count the number of letters in 'test'

return or return() signals end of the function. Here, it does not do anything other than to indicate there are no 'return values' and it will return the default 'None'.

Considered good practice to include it.

# Arguments and parameters

```
def length_of_word (word):      #2
    length = (len(word))
    print ("Word length = ", length)
    return
```

```
def main():
    word = input ("Enter a word: ") test
    length_of_word(word)        #1
```

```
main()
```

We are 'calling' the function and passing it an 'argument'.  
The function receives the argument as a 'parameter'  
i.e. 'argument' becomes 'parameter'

#1: **word** is a var holding string 'test';  
We call a function using the var name to pass it string 'test';  
**word** has become **word** where **word** is the **argument** containing the string 'test' being passed by the caller to the function  
#2: '**word**' is the function **parameter**  
When received by the function, **word** as **argument** becomes **word** as **parameter** in parentheses in the function header: It uses **word** holding string 'test' inside the function.

A parameter is a user-defined name in the function header and can have the same name as the argument.

**Only use the parameter name in the function, not the argument name**

# Aliasing

## Parameters with different names

```
def length_of_word (word):  
    # 'len()' to count characters in a word  
    length = (len(word))  
    print ("Word length = ", length)  
    return
```

```
def main():  
    word = input ("Enter a word: ") test  
    length_of_word(word)
```

main()

Same parameter name as argument name is okay, but can make debugging harder.  
If there's an error, which 'word' is causing it?

```
def length_of_word (a): # 'a' is an alias  
    # 'len()' to count characters in a word  
    length = (len(a))  
    print ("Word length = ", length)  
    return
```

```
def main():  
    word = input ("Enter a word: ") test  
    length_of_word(word)
```

main()

Aliasing can make debugging easier;  
Argument `word` is passed to the function and received as parameter `a`  
The function only uses `a`.  
Note: `a` is a poor name to use

# Function local variables

A local var is the parameter used in the function

```
def double (localNum):  
    #confirm parameter 'localNum' = 9  
    print ("Value of 'localNum' =", localNum)  
    #multiply parameter by 2 to double it  
    localNum = localNum * 2  
    #confirm parameter 'localNum' = 18  
    print ("Value of 'localNum' =", localNum)  
    return  
  
def main():  
    num = 9 #static assignment  
    #confirm num = 9 before the call  
    print ("Value of num before the call =", num)  
    #call 'double' and pass argument 9  
    double (num)  
    #confirm num = 9 after the call  
    print ("Value of num after the call =", num)  
  
main()
```

main() calls def main():  
def main() calls double with  
argument num

def double(localNum): receives  
parameter 9

We use localNum inside the  
function – it is called a 'local  
variable'.

The local variable is changed  
from 9 to 18 inside the function.  
Function ends

On return to main, the original  
value is still = 9;  
The change to 18 is 'local' to the  
function. Value 18 does not exist  
outside the 'scope' of the  
function

# Local variables – mechanics

In the previous slide:

- `double()` is called from `def main()`
- argument `x=9` is passed to it
- `def double` changes parameter `'localNum'` to value 18
- Function ends and return to main where `num` is still = 9

In reality:

- A copy of `num=9` is taken at the call and the copy is passed to the function
- Inside the function, it is the copy that is being used (as local variable `localNum`), so changes only affect the copy, not the original which is outside of the function

# Parameters have 'positional behaviour'

#parameter x receives argument x (=9) and parameter y receives argument y (=20)

```
def change (x, y):
```

```
    x = 42      #change 9 to 42 – local to the function
```

```
    y = y + 5   #change 20 to 25 - local to the function
```

```
    print ("Value of parameters within the function after change =",x,y)
```

```
    return
```

Does not change x=9, y=20  
outside of the function

```
def main():
```

```
    x = 9
```

```
    y = 20
```

```
    print ("Value of arguments before the call =", x,y)
```

```
    change (x, y)
```

```
    print ("Value of arguments after the call =", x,y)
```

# outside the function, x=9 and y=20 they do not change

#inside the function they are x=42 and y=25

```
main()
```

pass arguments in same order  
as the parameters are in